

Fiche récapitulative – STA211

Complément `fr_Tanagra_Stacking_Python`
Stacking – Agrégation par méta-apprentissage avec Python

Tanagra Data Mining – R. Rakotomalala

15 juin 2026

Résumé

Le stacking combine plusieurs classifieurs hétérogènes via un méta-classifieur qui apprend à corriger leurs erreurs. Ce tutoriel l'implémente en Python avec scikit-learn sur un problème de discrimination binaire dans le plan.

Table des matières

1	Données	2
2	Classifieurs de base	2
2.1	Régression logistique	2
2.2	K plus proches voisins (KNN)	2
2.3	Arbre de décision	2
2.4	SVM (noyau RBF)	2
3	Stacking avec scikit-learn	3
3.1	StackingClassifier	3
3.2	Interprétation	3
4	Stacking manuel (compréhension)	3
5	Comparaison R vs Python	4
	Questions clés (QCM)	5

Données

Problème de discrimination binaire dans le plan. 2000 observations, 2 variables prédictives X_1, X_2 .
Frontière parabolique :

$$\text{Si } 0.1 \times X_2 > X_1^2 \implies \text{pos, sinon neg}$$

Préparation :

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

D = pd.read_csv("artificial2d_data2.txt", sep="\t")
y = np.where(D["Y"] == "pos", 1, 0)
X = D[["X1", "X2"]].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=1500, test_size=500,
    random_state=100
)
```

1500 observations en apprentissage, 500 en test.

Classifieurs de base

Régression logistique

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(max_iter=5000)
lr.fit(X_train, y_train)
print(f"LR: {lr.score(X_test, y_test):.3f}")
```

Accuracy test : **0.678** ($\approx 67.8\%$).

K plus proches voisins (KNN)

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
print(f"KNN: {knn.score(X_test, y_test):.3f}")
```

Accuracy test : **0.918**.

Arbre de décision

```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier(max_depth=5, random_state=0)
tree.fit(X_train, y_train)
print(f"Tree: {tree.score(X_test, y_test):.3f}")
```

Accuracy test : **0.942**.

SVM (noyau RBF)

```
from sklearn.svm import SVC
svm = SVC(probability=True, random_state=0)
svm.fit(X_train, y_train)
print(f"SVM: {svm.score(X_test, y_test):.3f}")
```

Accuracy test : **0.968**.

Stacking avec scikit-learn

StackingClassifier

```
from sklearn.ensemble import StackingClassifier

base_learners = [
    ('lr', LogisticRegression(max_iter=5000)),
    ('knn', KNeighborsClassifier(n_neighbors=5)),
    ('tree', DecisionTreeClassifier(max_depth=5, random_state=0)),
    ('svm', SVC(probability=True, random_state=0))
]

meta = LogisticRegression(max_iter=5000)

stack = StackingClassifier(
    estimators=base_learners,
    final_estimator=meta,
    cv=5,
    stack_method='predict_proba'
)
stack.fit(X_train, y_train)
print(f"Stacking: {stack.score(X_test, y_test):.3f}")
```

Accuracy test : **0.978**.

Interprétation

Le stacking améliore la performance par rapport aux classifieurs individuels :

Classifieur	Accuracy test	Écart au stacking
Régression logistique	0.678	−0.300
KNN ($k = 5$)	0.918	−0.060
Arbre de décision	0.942	−0.036
SVM (RBF)	0.968	−0.010
Stacking	0.978	—

À retenir. Le stacking (**0.978**) surpasse le meilleur classifieur individuel (**SVM 0.968**). Le méta-modèle apprend à compenser les erreurs résiduelles du SVM grâce aux autres classifieurs. L'amélioration est modeste ici car le SVM est déjà très performant sur ces données.

Stacking manuel (compréhension)

À titre pédagogique, on peut décomposer le stacking :

```
from sklearn.model_selection import KFold
import numpy as np
```

```
def stacking_manual(X, y, base_learners, meta_learner, n_folds=5):
    kf = KFold(n_splits=n_folds, shuffle=True, random_state=0)
    Z = np.zeros((len(y), len(base_learners)))

    # Etape 1 : predictions hors-pli (CV)
    for train_idx, val_idx in kf.split(X):
        X_fold, X_val = X[train_idx], X[val_idx]
        y_fold = y[train_idx]

        for k, (_, model) in enumerate(base_learners):
            model.fit(X_fold, y_fold)
            Z[val_idx, k] = model.predict_proba(X_val)[: , 1]

    # Etape 2 : meta-modele sur Z
    meta_learner.fit(Z, y)

    # Etape 3 : re-entraîner les base learners sur tout X
    for _, model in base_learners:
        model.fit(X, y)

    return meta_learner, base_learners
```

Cette implémentation manuelle reproduit exactement le comportement de `StackingClassifier`.

Comparaison R vs Python

Aspect	R	Python
Stacking	<code>stacking::stacking()</code>	<code>ensemble.StackingClassifier</code>
Méta-modèle	<code>glm</code> / <code>rpart</code>	<code>LogisticRegression</code> / <code>Ridge</code>
CV	param. cv (défaut 5)	param. cv (défaut 5)
Syntaxe	<code>%>%</code>	API orientée objet

En R, le package `stacking` de Tanagra offre une interface similaire. La logique métier est identique : générer les prédictions niveau 1 par CV, puis appliquer le méta-modèle.

Questions clés (QCM)

1. Quelle classe scikit-learn implémente le stacking ? **R. StackingClassifier.**
2. Combien de classifieurs de base sont utilisés dans ce tutoriel ? **R. 4 (LR, KNN, Tree, SVM).**
3. Quelle est l'accuracy du stacking en test ? **R. 0.978 (97.8 %).**
4. Quel classifieur individuel a la meilleure performance ? **R. SVM (0.968).**
5. Quelle méthode génère les données d'entrée du méta-modèle ? **R. La validation croisée (CV).**
6. Quel méta-modèle est utilisé par défaut ? **R. Régression logistique.**
7. À quoi sert le paramètre `stack_method` ? **R. À choisir entre probas et classes prédites.**
8. Le stacking réduit-il le biais ou la variance ? **R. Les deux (biais + variance).**
9. Quelle est la principale différence entre Voting et Stacking ? **R. Stacking apprend à combiner.**
10. Que fait `passthrough=True` ? **R. Ajoute X original aux prédictions niveau 1.**